Student Name :Kanhaiya Uttam Naik

Batch :CS02

Roll Number :CS37

Aim

To implement graph-based clustering using Minimum Spanning Tree (MST) on the Breast Cancer Wisconsin (Diagnostic) dataset in Python and visualize the resulting clusters.

Objectives

1. To understand the concept of graph-based clustering and Minimum Spanning Trees (MST).
2. To load and preprocess the Breast Cancer Wisconsin dataset (scaling features and handling metadata).
3. To construct a distance graph from tumor feature representations using Euclidean distance.
4. To compute the Minimum Spanning Tree using Prim's or Kruskal's algorithm (or via scipy/networkx).
5. To form $K=2$ clusters (Malignant vs. Benign) by removing the single ($K - 1$) largest edge weight from the MST.
6. To evaluate the performance of the MST-based clustering model using evaluation metrics.

Relevant Theory

Graph-Based Clustering & Minimum Spanning Tree (MST)

Graph-based clustering represents data points as nodes in a graph, with edges representing relationships or distance metrics between individual samples.

- **Complete Graph Construction**: Given a dataset $X = \{x_1, x_2, \dots, x_n\}$, each tissue sample is treated as a vertex. An edge weight $w(i, j)$

corresponds to the distance between sample i and sample j (typically Euclidean distance on scaled features):$$w(i, j) = \Vert{}x_i - x_j\ Vert{}_2$$

- **Minimum Spanning Tree (MST)**: An MST is a subset of edges connecting all vertices in the graph without forming any cycles, while minimizing the total edge weight sum.
- **MST-Based Clustering Method**:
  1. Construct a fully connected graph with pairwise edge weights across standardized features.
  2. Compute the Minimum Spanning Tree (MST).
  3. To partition the data into K distinct clusters (such as Malignant and Benign groups), identify and remove the $K - 1$ largest (most expensive) edges from the MST.4. The resulting disconnected subgraphs represent the separate data clusters. ### Dataset Information Get the Breast Cancer Wisconsin (Diagnostic) dataset from Kaggle or via scikit-learn (load_breast_cancer). The dataset contains key cell nuclei features computed from digitized images of fine needle aspirates (FNA) of breast masses:
     - Total Instances: 569 samples
     - Target Classes: Malignant (M) and Benign (B)* Feature Attributes: 30 numerical features (e.g., mean radius, mean texture, mean perimeter, mean area, mean smoothness)

For 2D visualization, two key features (such as mean radius and mean texture) will be selected.

Task 1: Import Libraries and Load Dataset

- Import required libraries (numpy, pandas, matplotlib, scipy, networkx, sklearn).
- Load the Breast Cancer dataset using below link from Kaggle.

  https://www.kaggle.com/datasets/utkarshx27/breast-cancer-wisconsin-diagnostic-dataset

---

In [4]:
```
# Task 1: Import Libraries and Load Dataset
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import scipy
import networkx as nx
import io

from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, confusion_matrix, classification_report
)

# Upload dataset
print("Upload your brca.csv file")
uploaded = files.upload()

# Load the uploaded CSV file
file_name = next(iter(uploaded))
df = pd.read_csv(io.BytesIO(uploaded[file_name]))

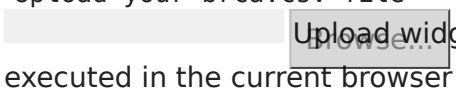
# Display first 5 rows
print("First 5 rows of the dataset:")
print(df.head())

# Display dataset information
print("\nDataset Shape:", df.shape)

print("\nDataset Information:")
df.info()

print("\nColumn Names:")
print(df.columns.tolist())
```

```
Upload your brca.csv file
```



Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving brca.csv to brca (1).csv
First 5 rows of the dataset:
   Unnamed: 0  x.radius_mean  x.texture_mean  x.perimeter_mean  x.area_me
an  \
0           1         13.540           14.36             87.46        56
6.3
1           2         13.080           15.71             85.63        52
0.0
2           3          9.504           12.44             60.34        27
3.9
3           4         13.030           18.42             82.61        52
3.8
4           5          8.196           16.84             51.71        20
1.9

   x.smoothness_mean  x.compactness_mean  x.concavity_mean  \
0            0.09779             0.08129           0.06664
1            0.10750             0.12700           0.04568
2            0.10240             0.06492           0.02956
3            0.08983             0.03766           0.02562
4            0.08600             0.05943           0.01588

   x.concave_pts_mean  x.symmetry_mean  ...  x.texture_worst  \
0            0.047810           0.1885  ...            19.26
1            0.031100           0.1967  ...            20.49
2            0.020760           0.1815  ...            15.66
3            0.029230           0.1467  ...            22.81
4            0.005917           0.1769  ...            21.96

   x.perimeter_worst  x.area_worst  x.smoothness_worst  x.compactness_wor
st  \
0              99.70         711.2             0.14400              0.177
30
1              96.09         630.5             0.13120              0.277
60
2              65.13         314.9             0.13240              0.114
80
3              84.46         545.9             0.09701              0.046
19
4              57.26         242.2             0.12970              0.135
70

   x.concavity_worst  x.concave_pts_worst  x.symmetry_worst  \
0            0.23900              0.12880            0.2977
1            0.18900              0.07283            0.3184
2            0.08867              0.06227            0.2450
3            0.04833              0.05013            0.1987
4            0.06880              0.02564            0.3105

   x.fractal_dim_worst  y
0              0.07259  B
1              0.08183  B
2              0.07773  B
3              0.06169  B
4              0.07409  B
```

```
[5 rows x 32 columns]

Dataset Shape: (569, 32)

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 32 columns):
 #   Column               Non-Null Count  Dtype  
---  ------               --------------  -----  
 0   Unnamed: 0           569 non-null    int64  
 1   x.radius_mean        569 non-null    float64
 2   x.texture_mean       569 non-null    float64
 3   x.perimeter_mean     569 non-null    float64
 4   x.area_mean          569 non-null    float64
 5   x.smoothness_mean    569 non-null    float64
 6   x.compactness_mean   569 non-null    float64
 7   x.concavity_mean     569 non-null    float64
 8   x.concave_pts_mean   569 non-null    float64
 9   x.symmetry_mean      569 non-null    float64
 10  x.fractal_dim_mean   569 non-null    float64
 11  x.radius_se          569 non-null    float64
 12  x.texture_se         569 non-null    float64
 13  x.perimeter_se       569 non-null    float64
 14  x.area_se            569 non-null    float64
 15  x.smoothness_se      569 non-null    float64
 16  x.compactness_se     569 non-null    float64
 17  x.concavity_se       569 non-null    float64
 18  x.concave_pts_se     569 non-null    float64
 19  x.symmetry_se        569 non-null    float64
 20  x.fractal_dim_se     569 non-null    float64
 21  x.radius_worst       569 non-null    float64
 22  x.texture_worst      569 non-null    float64
 23  x.perimeter_worst    569 non-null    float64
 24  x.area_worst         569 non-null    float64
 25  x.smoothness_worst   569 non-null    float64
 26  x.compactness_worst  569 non-null    float64
 27  x.concavity_worst    569 non-null    float64
 28  x.concave_pts_worst  569 non-null    float64
 29  x.symmetry_worst     569 non-null    float64
 30  x.fractal_dim_worst  569 non-null    float64
 31  y                    569 non-null    object 
dtypes: float64(30), int64(1), object(1)
memory usage: 142.4+ KB

Column Names:
['Unnamed: 0', 'x.radius_mean', 'x.texture_mean', 'x.perimeter_mean',
'x.area_mean', 'x.smoothness_mean', 'x.compactness_mean', 'x.concavity_me
an', 'x.concave_pts_mean', 'x.symmetry_mean', 'x.fractal_dim_mean', 'x.ra
dius_se', 'x.texture_se', 'x.perimeter_se', 'x.area_se', 'x.smoothness_s
e', 'x.compactness_se', 'x.concavity_se', 'x.concave_pts_se', 'x.symmetr
y_se', 'x.fractal_dim_se', 'x.radius_worst', 'x.texture_worst', 'x.perime
ter_worst', 'x.area_worst', 'x.smoothness_worst', 'x.compactness_worst',
'x.concavity_worst', 'x.concave_pts_worst', 'x.symmetry_worst', 'x.fracta
```

```
l_dim_worst', 'y']
```

Task 2: Explore the Dataset

- Display missing value checks, summary statistics, and feature dimensions.
- Check the diagnosis class distribution (Malignant vs. Benign).

In [5]:
```python
# Task 2: Explore the Dataset

# Check missing values
print("Missing Values in Each Column:")
print(df.isnull().sum())

# Display summary statistics
print("\nSummary Statistics:")
print(df.describe())

# Display feature dimensions
print("\nDataset Dimensions:", df.shape)

# Diagnosis class distribution
print("\nDiagnosis Class Distribution:")
print(df['y'].value_counts())

# Diagnosis distribution in percentage
print("\nDiagnosis Class Distribution (%):")
print(df['y'].value_counts(normalize=True) * 100)

# Plot class distribution
df['y'].value_counts().plot(kind='bar')
plt.title("Diagnosis Class Distribution")
plt.xlabel("Diagnosis")
plt.ylabel("Number of Samples")
plt.show()
```

```
Missing Values in Each Column:
Unnamed: 0              0
x.radius_mean           0
x.texture_mean          0
x.perimeter_mean        0
x.area_mean             0
x.smoothness_mean       0
x.compactness_mean      0
x.concavity_mean        0
x.concave_pts_mean      0
x.symmetry_mean         0
x.fractal_dim_mean      0
x.radius_se             0
x.texture_se            0
x.perimeter_se          0
x.area_se               0
x.smoothness_se         0
x.compactness_se        0
x.concavity_se          0
x.concave_pts_se        0
x.symmetry_se           0
x.fractal_dim_se        0
x.radius_worst          0
x.texture_worst         0
x.perimeter_worst       0
x.area_worst            0
x.smoothness_worst      0
x.compactness_worst     0
x.concavity_worst       0
x.concave_pts_worst     0
x.symmetry_worst        0
x.fractal_dim_worst     0
y                       0
dtype: int64

Summary Statistics:
       Unnamed: 0  x.radius_mean  x.texture_mean  x.perimeter_mean  \
count  569.000000     569.000000      569.000000        569.000000
mean   285.000000      14.127292       19.289649         91.969033
std    164.400426       3.524049        4.301036         24.298981
min      1.000000       6.981000        9.710000         43.790000
25%    143.000000      11.700000       16.170000         75.170000
50%    285.000000      13.370000       18.840000         86.240000
75%    427.000000      15.780000       21.800000        104.100000
max    569.000000      28.110000       39.280000        188.500000

       x.area_mean  x.smoothness_mean  x.compactness_mean  x.concavity_me
an  \
count   569.000000         569.000000          569.000000        569.0000
00
mean    654.889104           0.096360            0.104341          0.0887
99
std     351.914129           0.014064            0.052813          0.0797
20
min     143.500000           0.052630            0.019380          0.0000
```

```
00
25%     420.300000           0.086370           0.064920          0.0295
60
50%     551.100000           0.095870           0.092630          0.0615
40
75%     782.700000           0.105300           0.130400          0.1307
00
max    2501.000000           0.163400           0.345400          0.4268
00

       x.concave_pts_mean  x.symmetry_mean  ...  x.radius_worst  \
count          569.000000       569.000000  ...      569.000000
mean             0.048919         0.181162  ...       16.269190
std              0.038803         0.027414  ...        4.833242
min              0.000000         0.106000  ...        7.930000
25%              0.020310         0.161900  ...       13.010000
50%              0.033500         0.179200  ...       14.970000
75%              0.074000         0.195700  ...       18.790000
max              0.201200         0.304000  ...       36.040000

       x.texture_worst  x.perimeter_worst  x.area_worst  x.smoothness_wor
st  \
count       569.000000         569.000000    569.000000          569.0000
00
mean         25.677223         107.261213    880.583128            0.1323
69
std           6.146258          33.602542    569.356993            0.0228
32
min          12.020000          50.410000    185.200000            0.0711
70
25%          21.080000          84.110000    515.300000            0.1166
00
50%          25.410000          97.660000    686.500000            0.1313
00
75%          29.720000         125.400000   1084.000000            0.1460
00
max          49.540000         251.200000   4254.000000            0.2226
00

       x.compactness_worst  x.concavity_worst  x.concave_pts_worst  \
count           569.000000         569.000000           569.000000
mean              0.254265           0.272188             0.114606
std               0.157336           0.208624             0.065732
min               0.027290           0.000000             0.000000
25%               0.147200           0.114500             0.064930
50%               0.211900           0.226700             0.099930
75%               0.339100           0.382900             0.161400
max               1.058000           1.252000             0.291000

       x.symmetry_worst  x.fractal_dim_worst
count        569.000000           569.000000
mean           0.290076             0.083946
std            0.061867             0.018061
min            0.156500             0.055040
25%            0.250400             0.071460
```

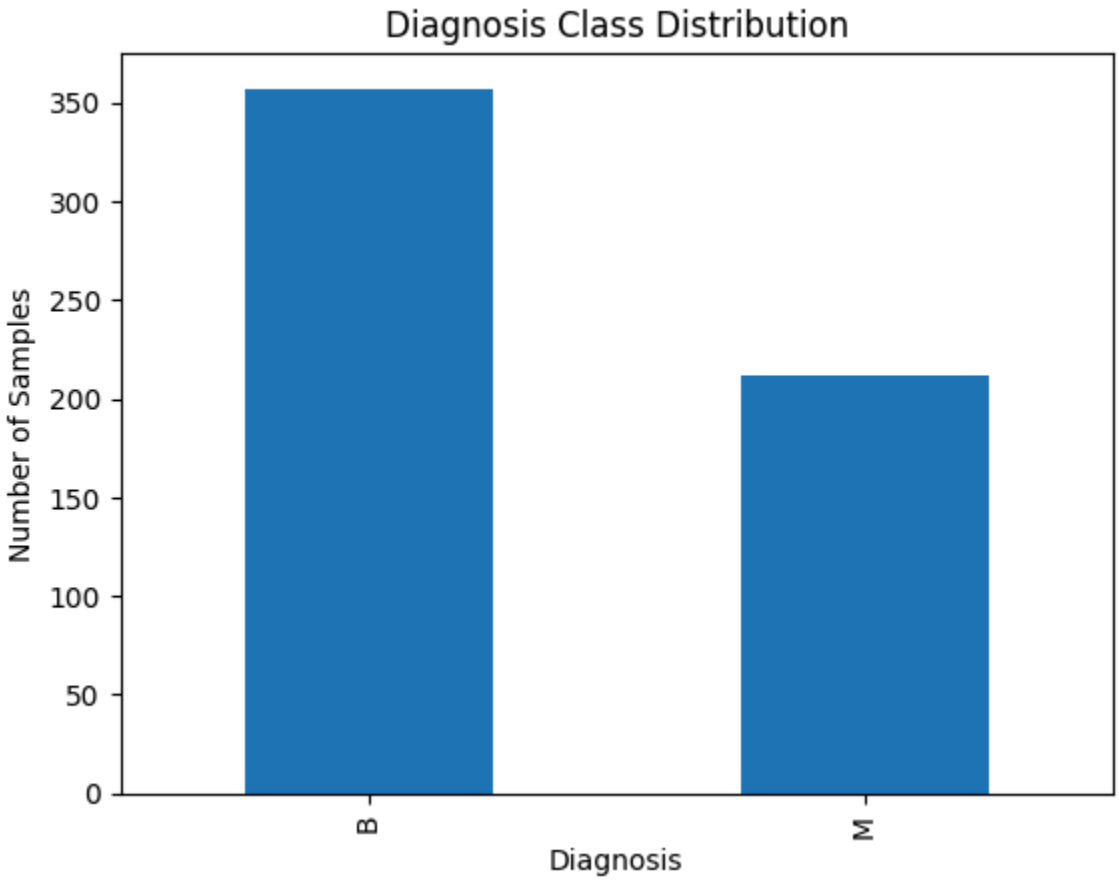
```
50%            0.282200             0.080040
75%            0.317900             0.092080
max            0.663800             0.207500

[8 rows x 31 columns]

Dataset Dimensions: (569, 32)

Diagnosis Class Distribution:
y
B    357
M    212
Name: count, dtype: int64

Diagnosis Class Distribution (%):
y
B    62.741652
M    37.258348
Name: proportion, dtype: float64
```



Task 3: Preprocess and Scale Features

- Select numerical features and drop non-informative identifiers (such as

id).

- Apply StandardScaler to normalize feature magnitudes prior to graph distance calculation.

In [6]:
```
# Task 3: Preprocess and Scale Features

from sklearn.preprocessing import StandardScaler

# Select numerical features
X = df.drop(['Unnamed: 0', 'y'], axis=1)

# Store diagnosis separately
y = df['y']

# Apply StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Convert scaled data back to DataFrame
X_scaled = pd.DataFrame(
    X_scaled,
    columns=X.columns,
    index=X.index
)

# Display results
print("Original Feature Dimensions:", X.shape)
print("Scaled Feature Dimensions:", X_scaled.shape)

print("\nFirst 5 Rows of Scaled Features:")
print(X_scaled.head())

print("\nMean of Scaled Features:")
print(X_scaled.mean().round(2))

print("\nStandard Deviation of Scaled Features:")
print(X_scaled.std().round(2))
```

```
Original Feature Dimensions: (569, 30)
Scaled Feature Dimensions: (569, 30)

First 5 Rows of Scaled Features:
   x.radius_mean  x.texture_mean  x.perimeter_mean  x.area_mean  \
0      -0.166799       -1.147162         -0.185728    -0.251957
1      -0.297446       -0.833008         -0.261106    -0.383638
2      -1.313080       -1.593959         -1.302806    -1.083572
3      -0.311646       -0.202373         -0.385500    -0.372831
4      -1.684571       -0.570050         -1.658278    -1.288347

   x.smoothness_mean  x.compactness_mean  x.concavity_mean  \
0           0.101747           -0.436850         -0.278210
1           0.792763            0.429422         -0.541362
2           0.429819           -0.747086         -0.743748
3          -0.464730           -1.263703         -0.793214
4          -0.737294           -0.851130         -0.915500

   x.concave_pts_mean  x.symmetry_mean  x.fractal_dim_mean  ...  \
0           -0.028609         0.267911           -0.728310  ...
1           -0.459627         0.567289            0.753087  ...
2           -0.726337         0.012345            0.886341  ...
3           -0.507861        -1.258183           -0.590802  ...
4           -1.109197        -0.155598            0.316465  ...

   x.radius_worst  x.texture_worst  x.perimeter_worst  x.area_worst  \
0       -0.240048        -1.045005          -0.225217     -0.297761
1       -0.366368        -0.844707          -0.332744     -0.439624
2       -1.250611        -1.631243          -1.254913     -0.994422
3       -0.614867        -0.466909          -0.679153     -0.588344
4       -1.512777        -0.605327          -1.489328     -1.122222

   x.smoothness_worst  x.compactness_worst  x.concavity_worst  \
0            0.509873            -0.489605          -0.159223
1           -0.051226             0.148443          -0.399099
2            0.001377            -0.887193          -0.880434
3           -1.549975            -1.323648          -1.073966
4           -0.116980            -0.754239          -0.975761

   x.concave_pts_worst  x.symmetry_worst  x.fractal_dim_worst
0             0.216123          0.123347           -0.629292
1            -0.636110          0.458227           -0.117250
2            -0.796903         -0.729224           -0.344455
3            -0.981753         -1.478256           -1.233324
4            -1.354653          0.330422           -0.546168

[5 rows x 30 columns]

Mean of Scaled Features:
x.radius_mean         -0.0
x.texture_mean        -0.0
x.perimeter_mean      -0.0
x.area_mean           -0.0
x.smoothness_mean      0.0
x.compactness_mean     0.0
```

```
x.concavity_mean       0.0
x.concave_pts_mean     0.0
x.symmetry_mean        0.0
x.fractal_dim_mean     0.0
x.radius_se            0.0
x.texture_se          -0.0
x.perimeter_se         0.0
x.area_se             -0.0
x.smoothness_se       -0.0
x.compactness_se       0.0
x.concavity_se        -0.0
x.concave_pts_se       0.0
x.symmetry_se          0.0
x.fractal_dim_se      -0.0
x.radius_worst        -0.0
x.texture_worst        0.0
x.perimeter_worst      0.0
x.area_worst          -0.0
x.smoothness_worst    -0.0
x.compactness_worst    0.0
x.concavity_worst      0.0
x.concave_pts_worst    0.0
x.symmetry_worst       0.0
x.fractal_dim_worst    0.0
dtype: float64

Standard Deviation of Scaled Features:
x.radius_mean          1.0
x.texture_mean         1.0
x.perimeter_mean       1.0
x.area_mean            1.0
x.smoothness_mean      1.0
x.compactness_mean     1.0
x.concavity_mean       1.0
x.concave_pts_mean     1.0
x.symmetry_mean        1.0
x.fractal_dim_mean     1.0
x.radius_se            1.0
x.texture_se           1.0
x.perimeter_se         1.0
x.area_se              1.0
x.smoothness_se        1.0
x.compactness_se       1.0
x.concavity_se         1.0
x.concave_pts_se       1.0
x.symmetry_se          1.0
x.fractal_dim_se       1.0
x.radius_worst         1.0
x.texture_worst        1.0
x.perimeter_worst      1.0
x.area_worst           1.0
x.smoothness_worst     1.0
x.compactness_worst    1.0
x.concavity_worst      1.0
x.concave_pts_worst    1.0
```

```
x.symmetry_worst       1.0
x.fractal_dim_worst    1.0
dtype: float64
```

Task 4: Construct Pairwise Distance Matrix & Graph

- Compute pairwise Euclidean distance matrix across all scaled samples using scipy.spatial.distance.pdist or cdist.
- Represent the dataset as a weighted complete graph.

In [7]:
```python
# Task 4: Construct Pairwise Distance Matrix & Graph

from scipy.spatial.distance import pdist, squareform
import networkx as nx

# Compute pairwise Euclidean distances
distance_matrix = squareform(
    pdist(X_scaled, metric='euclidean')
)

# Display distance matrix dimensions
print("Distance Matrix Shape:", distance_matrix.shape)

# Display first 5x5 portion
print("\nFirst 5x5 Distance Matrix:")
print(distance_matrix[:5, :5])

# Create a weighted complete graph
G = nx.complete_graph(len(X_scaled))

# Add Euclidean distances as edge weights
for i, j in G.edges():
    G[i][j]['weight'] = distance_matrix[i, j]

# Display graph information
print("\nGraph Information:")
print("Number of Nodes:", G.number_of_nodes())
print("Number of Edges:", G.number_of_edges())
```

```
Distance Matrix Shape: (569, 569)

First 5x5 Distance Matrix:
[[0.         3.10724668 3.62314178 5.20448164 4.46708561]
 [3.10724668 0.         4.01027585 5.62302017 4.37098277]
 [3.62314178 4.01027585 0.         5.12193591 2.94839575]
 [5.20448164 5.62302017 5.12193591 0.         5.02798792]
 [4.46708561 4.37098277 2.94839575 5.02798792 0.        ]]

Graph Information:
Number of Nodes: 569
Number of Edges: 161596
```

Task 5: Compute the Minimum Spanning Tree (MST)

- Compute the MST using scipy.sparse.csgraph.minimum_spanning_tree or networkx.minimum_spanning_tree.
- Extract the resulting graph connections and weights.

In [8]:
```python
# Task 5: Compute the Minimum Spanning Tree (MST)

# Compute the Minimum Spanning Tree
MST = nx.minimum_spanning_tree(G, weight='weight')

# Display MST information
print("Minimum Spanning Tree (MST)")
print("Number of Nodes:", MST.number_of_nodes())
print("Number of Edges:", MST.number_of_edges())

# Extract MST connections and weights
print("\nMST Connections and Weights:")

for u, v, data in MST.edges(data=True):
    print(
        f"Node {u} -- Node {v} : "
        f"Weight = {data['weight']:.4f}"
    )

# Calculate total MST weight
total_weight = MST.size(weight='weight')

print("\nTotal MST Weight:", round(total_weight, 4))
```

Minimum Spanning Tree (MST)
Number of Nodes: 569
Number of Edges: 568

MST Connections and Weights:
Node 0 -- Node 81 : Weight = 1.6886
Node 0 -- Node 108 : Weight = 2.1714
Node 1 -- Node 105 : Weight = 1.6607
Node 2 -- Node 70 : Weight = 2.2189
Node 3 -- Node 170 : Weight = 2.3462
Node 3 -- Node 222 : Weight = 2.5415
Node 4 -- Node 279 : Weight = 1.8603
Node 4 -- Node 84 : Weight = 2.1660
Node 5 -- Node 125 : Weight = 1.7659
Node 5 -- Node 172 : Weight = 2.0171
Node 6 -- Node 336 : Weight = 1.4204
Node 6 -- Node 393 : Weight = 1.6933
Node 7 -- Node 140 : Weight = 1.7730
Node 8 -- Node 66 : Weight = 1.5108
Node 9 -- Node 126 : Weight = 1.5756
Node 9 -- Node 192 : Weight = 1.6264
Node 9 -- Node 95 : Weight = 1.6729
Node 10 -- Node 126 : Weight = 1.7781
Node 11 -- Node 140 : Weight = 1.3252
Node 11 -- Node 192 : Weight = 1.8396
Node 12 -- Node 322 : Weight = 1.9960
Node 13 -- Node 191 : Weight = 3.1979
Node 13 -- Node 247 : Weight = 3.2457
Node 14 -- Node 16 : Weight = 2.7457
Node 15 -- Node 103 : Weight = 3.6315
Node 16 -- Node 24 : Weight = 1.7684
Node 16 -- Node 182 : Weight = 2.2230
Node 16 -- Node 240 : Weight = 2.8880
Node 17 -- Node 19 : Weight = 1.8308
Node 18 -- Node 208 : Weight = 7.7911
Node 18 -- Node 69 : Weight = 10.4761
Node 19 -- Node 174 : Weight = 1.3574
Node 19 -- Node 117 : Weight = 1.5689
Node 19 -- Node 90 : Weight = 1.5768
Node 20 -- Node 85 : Weight = 5.2568
Node 21 -- Node 57 : Weight = 1.1487
Node 21 -- Node 23 : Weight = 1.2304
Node 21 -- Node 226 : Weight = 1.2326
Node 21 -- Node 173 : Weight = 1.2485
Node 21 -- Node 101 : Weight = 1.3529
Node 21 -- Node 293 : Weight = 1.4312
Node 21 -- Node 107 : Weight = 1.5423
Node 22 -- Node 186 : Weight = 3.5225
Node 23 -- Node 201 : Weight = 1.0061
Node 23 -- Node 204 : Weight = 1.4127
Node 23 -- Node 338 : Weight = 1.7955
Node 25 -- Node 470 : Weight = 1.3914
Node 26 -- Node 129 : Weight = 1.3833
Node 26 -- Node 72 : Weight = 1.5140
Node 27 -- Node 96 : Weight = 1.8901

Node 27 -- Node 262 : Weight = 2.1122
Node 28 -- Node 485 : Weight = 2.3830
Node 28 -- Node 53 : Weight = 2.4396
Node 29 -- Node 339 : Weight = 1.1209
Node 29 -- Node 336 : Weight = 1.7064
Node 29 -- Node 120 : Weight = 2.0057
Node 30 -- Node 258 : Weight = 1.9259
Node 30 -- Node 299 : Weight = 2.9027
Node 31 -- Node 201 : Weight = 1.2858
Node 31 -- Node 302 : Weight = 1.7737
Node 32 -- Node 201 : Weight = 2.3672
Node 32 -- Node 502 : Weight = 2.4275
Node 33 -- Node 155 : Weight = 2.4116
Node 34 -- Node 132 : Weight = 1.2732
Node 34 -- Node 49 : Weight = 1.3423
Node 34 -- Node 241 : Weight = 1.4390
Node 34 -- Node 190 : Weight = 1.8475
Node 35 -- Node 106 : Weight = 3.2766
Node 36 -- Node 200 : Weight = 1.6890
Node 36 -- Node 271 : Weight = 1.7935
Node 37 -- Node 330 : Weight = 2.0180
Node 37 -- Node 39 : Weight = 2.1348
Node 37 -- Node 96 : Weight = 2.2170
Node 38 -- Node 281 : Weight = 1.7528
Node 38 -- Node 345 : Weight = 2.3905
Node 39 -- Node 394 : Weight = 2.3039
Node 39 -- Node 280 : Weight = 3.0771
Node 40 -- Node 304 : Weight = 1.2106
Node 41 -- Node 47 : Weight = 2.1662
Node 42 -- Node 171 : Weight = 2.6927
Node 42 -- Node 82 : Weight = 2.8222
Node 42 -- Node 187 : Weight = 2.8894
Node 43 -- Node 194 : Weight = 2.7468
Node 43 -- Node 143 : Weight = 5.6238
Node 44 -- Node 208 : Weight = 4.4136
Node 45 -- Node 171 : Weight = 2.1177
Node 46 -- Node 322 : Weight = 1.6533
Node 47 -- Node 338 : Weight = 1.9109
Node 48 -- Node 334 : Weight = 4.5984
Node 48 -- Node 468 : Weight = 8.8264
Node 50 -- Node 267 : Weight = 1.7888
Node 50 -- Node 105 : Weight = 1.8810
Node 51 -- Node 224 : Weight = 2.0695
Node 52 -- Node 203 : Weight = 1.3332
Node 52 -- Node 196 : Weight = 1.4218
Node 52 -- Node 136 : Weight = 1.4857
Node 53 -- Node 244 : Weight = 3.0537
Node 54 -- Node 185 : Weight = 1.7959
Node 55 -- Node 232 : Weight = 1.2724
Node 55 -- Node 65 : Weight = 1.7596
Node 55 -- Node 311 : Weight = 1.8590
Node 56 -- Node 502 : Weight = 2.0780
Node 58 -- Node 334 : Weight = 2.7806
Node 58 -- Node 134 : Weight = 3.6946
Node 59 -- Node 84 : Weight = 2.0132

Node 60 -- Node 182 : Weight = 1.7042
Node 60 -- Node 327 : Weight = 1.7189
Node 61 -- Node 204 : Weight = 1.3995
Node 62 -- Node 251 : Weight = 1.6213
Node 63 -- Node 171 : Weight = 2.9348
Node 64 -- Node 233 : Weight = 2.4634
Node 65 -- Node 294 : Weight = 2.3729
Node 66 -- Node 291 : Weight = 1.3684
Node 66 -- Node 217 : Weight = 1.4740
Node 66 -- Node 296 : Weight = 1.6013
Node 67 -- Node 191 : Weight = 2.6780
Node 68 -- Node 119 : Weight = 3.9170
Node 68 -- Node 333 : Weight = 4.0321
Node 70 -- Node 81 : Weight = 1.7162
Node 70 -- Node 135 : Weight = 1.8251
Node 71 -- Node 147 : Weight = 1.5490
Node 71 -- Node 177 : Weight = 2.2430
Node 72 -- Node 226 : Weight = 1.3942
Node 72 -- Node 212 : Weight = 1.5920
Node 72 -- Node 148 : Weight = 1.6140
Node 72 -- Node 255 : Weight = 1.6358
Node 72 -- Node 243 : Weight = 1.6526
Node 73 -- Node 128 : Weight = 2.9102
Node 74 -- Node 149 : Weight = 1.2180
Node 74 -- Node 173 : Weight = 1.3380
Node 74 -- Node 227 : Weight = 1.4053
Node 74 -- Node 132 : Weight = 1.5325
Node 74 -- Node 150 : Weight = 1.7077
Node 74 -- Node 79 : Weight = 2.3290
Node 75 -- Node 179 : Weight = 1.7905
Node 76 -- Node 127 : Weight = 2.2201
Node 76 -- Node 195 : Weight = 2.4678
Node 77 -- Node 330 : Weight = 2.1995
Node 78 -- Node 136 : Weight = 1.4449
Node 80 -- Node 293 : Weight = 1.5375
Node 81 -- Node 132 : Weight = 1.6482
Node 83 -- Node 135 : Weight = 2.0669
Node 84 -- Node 159 : Weight = 2.3805
Node 85 -- Node 145 : Weight = 5.2961
Node 85 -- Node 295 : Weight = 5.3825
Node 86 -- Node 131 : Weight = 2.2786
Node 87 -- Node 175 : Weight = 1.8181
Node 88 -- Node 139 : Weight = 2.4822
Node 89 -- Node 279 : Weight = 1.9662
Node 91 -- Node 242 : Weight = 1.7391
Node 91 -- Node 178 : Weight = 2.6633
Node 92 -- Node 95 : Weight = 1.5763
Node 92 -- Node 252 : Weight = 1.7380
Node 93 -- Node 502 : Weight = 3.2027
Node 94 -- Node 284 : Weight = 5.4884
Node 95 -- Node 268 : Weight = 1.3686
Node 96 -- Node 97 : Weight = 1.6820
Node 97 -- Node 320 : Weight = 1.5712
Node 97 -- Node 317 : Weight = 1.7372
Node 97 -- Node 328 : Weight = 2.1176

Node 97 -- Node 223 : Weight = 2.1208
Node 97 -- Node 309 : Weight = 2.2082
Node 98 -- Node 106 : Weight = 1.6003
Node 98 -- Node 182 : Weight = 1.7356
Node 98 -- Node 133 : Weight = 2.3840
Node 98 -- Node 350 : Weight = 2.6108
Node 99 -- Node 396 : Weight = 2.4028
Node 100 -- Node 283 : Weight = 1.9291
Node 101 -- Node 290 : Weight = 1.2300
Node 101 -- Node 188 : Weight = 1.5199
Node 101 -- Node 156 : Weight = 1.6009
Node 102 -- Node 195 : Weight = 1.7833
Node 103 -- Node 345 : Weight = 2.5007
Node 104 -- Node 329 : Weight = 2.0603
Node 105 -- Node 292 : Weight = 1.7199
Node 105 -- Node 245 : Weight = 1.9085
Node 107 -- Node 254 : Weight = 1.4365
Node 107 -- Node 276 : Weight = 1.4519
Node 109 -- Node 340 : Weight = 1.3772
Node 109 -- Node 219 : Weight = 1.7228
Node 110 -- Node 283 : Weight = 1.7754
Node 110 -- Node 183 : Weight = 1.8336
Node 111 -- Node 278 : Weight = 2.4903
Node 112 -- Node 274 : Weight = 2.4719
Node 113 -- Node 273 : Weight = 2.5208
Node 114 -- Node 190 : Weight = 1.8511
Node 115 -- Node 257 : Weight = 1.8141
Node 116 -- Node 269 : Weight = 2.7409
Node 117 -- Node 254 : Weight = 1.3684
Node 117 -- Node 256 : Weight = 1.6316
Node 117 -- Node 314 : Weight = 1.7661
Node 118 -- Node 158 : Weight = 1.6969
Node 119 -- Node 277 : Weight = 3.8301
Node 120 -- Node 228 : Weight = 2.3542
Node 121 -- Node 240 : Weight = 1.8956
Node 122 -- Node 222 : Weight = 1.9125
Node 123 -- Node 277 : Weight = 2.5180
Node 124 -- Node 250 : Weight = 2.2431
Node 125 -- Node 132 : Weight = 1.5439
Node 125 -- Node 245 : Weight = 1.6105
Node 125 -- Node 297 : Weight = 1.7200
Node 125 -- Node 289 : Weight = 1.7752
Node 127 -- Node 154 : Weight = 2.1620
Node 128 -- Node 200 : Weight = 1.6796
Node 129 -- Node 234 : Weight = 1.9303
Node 130 -- Node 171 : Weight = 2.1521
Node 131 -- Node 217 : Weight = 1.6012
Node 131 -- Node 160 : Weight = 1.6281
Node 132 -- Node 219 : Weight = 1.0971
Node 132 -- Node 326 : Weight = 1.5708
Node 133 -- Node 220 : Weight = 3.3518
Node 134 -- Node 325 : Weight = 3.7556
Node 136 -- Node 291 : Weight = 1.4606
Node 136 -- Node 163 : Weight = 1.5059
Node 137 -- Node 293 : Weight = 1.8932

Node 138 -- Node 197 : Weight = 2.9730
Node 139 -- Node 224 : Weight = 1.8935
Node 139 -- Node 194 : Weight = 2.2564
Node 140 -- Node 344 : Weight = 1.9567
Node 141 -- Node 265 : Weight = 2.4625
Node 141 -- Node 259 : Weight = 2.7790
Node 141 -- Node 218 : Weight = 3.8812
Node 142 -- Node 160 : Weight = 1.6627
Node 144 -- Node 234 : Weight = 1.9480
Node 144 -- Node 157 : Weight = 2.2106
Node 144 -- Node 261 : Weight = 3.4855
Node 146 -- Node 246 : Weight = 1.9282
Node 147 -- Node 185 : Weight = 1.6174
Node 148 -- Node 162 : Weight = 1.7690
Node 151 -- Node 251 : Weight = 2.2158
Node 152 -- Node 288 : Weight = 1.1477
Node 153 -- Node 347 : Weight = 2.1992
Node 153 -- Node 350 : Weight = 2.7853
Node 154 -- Node 224 : Weight = 2.2124
Node 154 -- Node 228 : Weight = 2.2180
Node 154 -- Node 233 : Weight = 2.5249
Node 154 -- Node 193 : Weight = 2.7616
Node 155 -- Node 231 : Weight = 1.9397
Node 155 -- Node 321 : Weight = 2.1195
Node 158 -- Node 203 : Weight = 1.5538
Node 160 -- Node 161 : Weight = 1.4898
Node 161 -- Node 260 : Weight = 1.6343
Node 164 -- Node 216 : Weight = 1.6698
Node 164 -- Node 241 : Weight = 1.8121
Node 165 -- Node 219 : Weight = 2.2023
Node 166 -- Node 178 : Weight = 5.1383
Node 167 -- Node 176 : Weight = 1.4922
Node 168 -- Node 192 : Weight = 1.7248
Node 169 -- Node 295 : Weight = 3.7942
Node 171 -- Node 253 : Weight = 2.1128
Node 171 -- Node 310 : Weight = 3.0772
Node 173 -- Node 203 : Weight = 1.4117
Node 173 -- Node 225 : Weight = 1.7574
Node 175 -- Node 196 : Weight = 1.8549
Node 176 -- Node 251 : Weight = 1.4787
Node 176 -- Node 252 : Weight = 1.7096
Node 179 -- Node 340 : Weight = 1.5400
Node 179 -- Node 319 : Weight = 1.7950
Node 180 -- Node 188 : Weight = 1.2694
Node 181 -- Node 241 : Weight = 2.3813
Node 182 -- Node 221 : Weight = 1.7111
Node 182 -- Node 242 : Weight = 2.1182
Node 183 -- Node 485 : Weight = 1.7454
Node 183 -- Node 246 : Weight = 1.7553
Node 183 -- Node 207 : Weight = 2.1485
Node 183 -- Node 510 : Weight = 2.3118
Node 184 -- Node 249 : Weight = 2.0915
Node 185 -- Node 289 : Weight = 1.7762
Node 185 -- Node 211 : Weight = 2.2941
Node 185 -- Node 322 : Weight = 2.4682

Node 186 -- Node 297 : Weight = 1.3602
Node 188 -- Node 298 : Weight = 1.6196
Node 188 -- Node 342 : Weight = 1.9954
Node 189 -- Node 232 : Weight = 1.9424
Node 191 -- Node 317 : Weight = 2.5234
Node 193 -- Node 197 : Weight = 2.9362
Node 195 -- Node 509 : Weight = 3.6391
Node 196 -- Node 257 : Weight = 1.4871
Node 198 -- Node 231 : Weight = 2.1466
Node 199 -- Node 252 : Weight = 1.4312
Node 200 -- Node 339 : Weight = 1.2998
Node 200 -- Node 301 : Weight = 1.5668
Node 200 -- Node 257 : Weight = 1.5894
Node 201 -- Node 236 : Weight = 1.6215
Node 202 -- Node 302 : Weight = 1.7514
Node 202 -- Node 556 : Weight = 1.8378
Node 202 -- Node 498 : Weight = 2.5236
Node 203 -- Node 268 : Weight = 1.4392
Node 203 -- Node 329 : Weight = 1.4964
Node 204 -- Node 275 : Weight = 1.3220
Node 204 -- Node 304 : Weight = 1.4002
Node 204 -- Node 286 : Weight = 1.6712
Node 205 -- Node 313 : Weight = 1.2997
Node 206 -- Node 268 : Weight = 1.3730
Node 206 -- Node 263 : Weight = 1.6324
Node 208 -- Node 295 : Weight = 4.3188
Node 209 -- Node 271 : Weight = 1.8607
Node 209 -- Node 274 : Weight = 1.8714
Node 210 -- Node 286 : Weight = 2.0075
Node 213 -- Node 265 : Weight = 1.8165
Node 214 -- Node 249 : Weight = 2.2421
Node 214 -- Node 285 : Weight = 2.4682
Node 215 -- Node 286 : Weight = 1.4077
Node 215 -- Node 283 : Weight = 1.6443
Node 216 -- Node 312 : Weight = 2.1440
Node 217 -- Node 288 : Weight = 1.4950
Node 221 -- Node 315 : Weight = 1.7135
Node 222 -- Node 257 : Weight = 1.5986
Node 223 -- Node 562 : Weight = 2.2461
Node 229 -- Node 304 : Weight = 1.5178
Node 230 -- Node 252 : Weight = 2.0089
Node 230 -- Node 300 : Weight = 2.4476
Node 231 -- Node 327 : Weight = 1.6674
Node 232 -- Node 314 : Weight = 1.5741
Node 235 -- Node 436 : Weight = 1.9359
Node 237 -- Node 342 : Weight = 2.5862
Node 238 -- Node 246 : Weight = 2.1229
Node 239 -- Node 315 : Weight = 1.7492
Node 243 -- Node 250 : Weight = 1.5942
Node 243 -- Node 281 : Weight = 1.7794
Node 246 -- Node 278 : Weight = 2.1462
Node 248 -- Node 319 : Weight = 1.2162
Node 249 -- Node 289 : Weight = 2.1938
Node 249 -- Node 253 : Weight = 2.2629
Node 250 -- Node 343 : Weight = 2.2485

Node 253 -- Node 334 : Weight = 2.1168
Node 254 -- Node 313 : Weight = 1.4342
Node 258 -- Node 313 : Weight = 1.3632
Node 259 -- Node 278 : Weight = 2.7853
Node 262 -- Node 354 : Weight = 2.4667
Node 263 -- Node 548 : Weight = 3.1457
Node 264 -- Node 556 : Weight = 1.6557
Node 264 -- Node 287 : Weight = 1.9972
Node 266 -- Node 348 : Weight = 1.5359
Node 266 -- Node 273 : Weight = 2.2194
Node 266 -- Node 270 : Weight = 2.7516
Node 266 -- Node 349 : Weight = 2.8451
Node 268 -- Node 324 : Weight = 1.6620
Node 269 -- Node 337 : Weight = 2.0644
Node 271 -- Node 272 : Weight = 1.0276
Node 271 -- Node 337 : Weight = 1.6919
Node 271 -- Node 346 : Weight = 1.8232
Node 277 -- Node 335 : Weight = 3.6714
Node 279 -- Node 342 : Weight = 2.2008
Node 279 -- Node 332 : Weight = 3.5146
Node 280 -- Node 331 : Weight = 2.7839
Node 280 -- Node 457 : Weight = 3.2091
Node 282 -- Node 343 : Weight = 2.8612
Node 284 -- Node 351 : Weight = 2.7588
Node 284 -- Node 355 : Weight = 3.3735
Node 287 -- Node 352 : Weight = 2.2897
Node 291 -- Node 493 : Weight = 1.7050
Node 292 -- Node 409 : Weight = 2.0591
Node 295 -- Node 546 : Weight = 4.2193
Node 297 -- Node 306 : Weight = 1.5417
Node 297 -- Node 317 : Weight = 1.6571
Node 298 -- Node 436 : Weight = 1.8817
Node 303 -- Node 409 : Weight = 1.9773
Node 305 -- Node 409 : Weight = 2.3021
Node 306 -- Node 315 : Weight = 1.7341
Node 307 -- Node 308 : Weight = 5.3611
Node 308 -- Node 310 : Weight = 5.7611
Node 310 -- Node 318 : Weight = 4.3382
Node 316 -- Node 320 : Weight = 2.4093
Node 321 -- Node 341 : Weight = 2.2379
Node 323 -- Node 397 : Weight = 1.9462
Node 323 -- Node 328 : Weight = 2.0214
Node 335 -- Node 390 : Weight = 2.1291
Node 336 -- Node 532 : Weight = 2.8407
Node 343 -- Node 351 : Weight = 3.3692
Node 344 -- Node 356 : Weight = 2.8033
Node 346 -- Node 348 : Weight = 1.6331
Node 349 -- Node 353 : Weight = 2.8927
Node 357 -- Node 411 : Weight = 4.8299
Node 358 -- Node 518 : Weight = 2.2871
Node 359 -- Node 561 : Weight = 2.4515
Node 359 -- Node 398 : Weight = 2.6763
Node 359 -- Node 411 : Weight = 3.7056
Node 360 -- Node 454 : Weight = 7.8710
Node 361 -- Node 550 : Weight = 2.8827

Node 362 -- Node 399 : Weight = 2.5029
Node 362 -- Node 365 : Weight = 2.6951
Node 362 -- Node 423 : Weight = 2.7035
Node 362 -- Node 385 : Weight = 3.0699
Node 363 -- Node 505 : Weight = 1.1010
Node 363 -- Node 494 : Weight = 1.4449
Node 363 -- Node 435 : Weight = 1.8163
Node 363 -- Node 484 : Weight = 1.8910
Node 363 -- Node 417 : Weight = 2.7728
Node 364 -- Node 552 : Weight = 2.3934
Node 364 -- Node 555 : Weight = 2.7031
Node 366 -- Node 371 : Weight = 5.3181
Node 367 -- Node 497 : Weight = 1.8699
Node 367 -- Node 401 : Weight = 2.1188
Node 368 -- Node 473 : Weight = 2.3028
Node 369 -- Node 395 : Weight = 6.9690
Node 370 -- Node 418 : Weight = 2.1220
Node 371 -- Node 474 : Weight = 2.9617
Node 371 -- Node 525 : Weight = 3.9325
Node 372 -- Node 474 : Weight = 2.9928
Node 372 -- Node 489 : Weight = 3.8877
Node 372 -- Node 563 : Weight = 3.9069
Node 373 -- Node 446 : Weight = 1.5321
Node 373 -- Node 451 : Weight = 1.8671
Node 374 -- Node 425 : Weight = 2.1151
Node 374 -- Node 528 : Weight = 2.3569
Node 374 -- Node 426 : Weight = 2.4731
Node 375 -- Node 541 : Weight = 2.2234
Node 375 -- Node 377 : Weight = 2.5259
Node 376 -- Node 546 : Weight = 3.8524
Node 376 -- Node 515 : Weight = 4.2661
Node 377 -- Node 443 : Weight = 3.3330
Node 378 -- Node 402 : Weight = 2.3462
Node 379 -- Node 411 : Weight = 4.3557
Node 380 -- Node 388 : Weight = 2.8008
Node 381 -- Node 536 : Weight = 2.6189
Node 382 -- Node 473 : Weight = 3.0761
Node 383 -- Node 438 : Weight = 2.1209
Node 384 -- Node 386 : Weight = 2.7080
Node 384 -- Node 486 : Weight = 3.1317
Node 386 -- Node 528 : Weight = 2.6379
Node 387 -- Node 408 : Weight = 2.8900
Node 387 -- Node 459 : Weight = 2.9720
Node 388 -- Node 473 : Weight = 2.1614
Node 389 -- Node 522 : Weight = 2.7245
Node 389 -- Node 473 : Weight = 2.8977
Node 390 -- Node 473 : Weight = 2.0950
Node 391 -- Node 502 : Weight = 4.5263
Node 392 -- Node 537 : Weight = 1.8093
Node 395 -- Node 519 : Weight = 6.3473
Node 396 -- Node 473 : Weight = 1.7813
Node 396 -- Node 403 : Weight = 1.9714
Node 396 -- Node 537 : Weight = 2.0929
Node 396 -- Node 470 : Weight = 2.1040
Node 397 -- Node 452 : Weight = 2.0517

Node 397 -- Node 537 : Weight = 2.0759
Node 397 -- Node 430 : Weight = 2.1637
Node 399 -- Node 555 : Weight = 1.8930
Node 399 -- Node 425 : Weight = 2.2399
Node 400 -- Node 549 : Weight = 2.6933
Node 401 -- Node 446 : Weight = 1.7859
Node 402 -- Node 484 : Weight = 2.1561
Node 402 -- Node 471 : Weight = 2.5453
Node 403 -- Node 555 : Weight = 1.9459
Node 404 -- Node 426 : Weight = 3.1643
Node 404 -- Node 534 : Weight = 3.3570
Node 404 -- Node 545 : Weight = 4.4342
Node 405 -- Node 406 : Weight = 1.9963
Node 405 -- Node 455 : Weight = 2.6239
Node 406 -- Node 508 : Weight = 1.8764
Node 407 -- Node 518 : Weight = 1.6715
Node 408 -- Node 482 : Weight = 3.8629
Node 409 -- Node 464 : Weight = 2.2716
Node 410 -- Node 444 : Weight = 2.7272
Node 412 -- Node 515 : Weight = 7.0895
Node 413 -- Node 559 : Weight = 4.2626
Node 414 -- Node 536 : Weight = 5.1013
Node 415 -- Node 549 : Weight = 1.8924
Node 415 -- Node 428 : Weight = 1.9536
Node 416 -- Node 526 : Weight = 2.8438
Node 418 -- Node 526 : Weight = 2.7044
Node 419 -- Node 448 : Weight = 2.1440
Node 420 -- Node 560 : Weight = 2.3200
Node 420 -- Node 471 : Weight = 2.5883
Node 420 -- Node 514 : Weight = 2.9205
Node 421 -- Node 562 : Weight = 1.8733
Node 422 -- Node 446 : Weight = 1.9635
Node 422 -- Node 526 : Weight = 2.0975
Node 422 -- Node 567 : Weight = 3.0038
Node 424 -- Node 568 : Weight = 5.8771
Node 424 -- Node 429 : Weight = 8.0780
Node 425 -- Node 475 : Weight = 2.6141
Node 425 -- Node 447 : Weight = 3.1763
Node 426 -- Node 530 : Weight = 3.1153
Node 426 -- Node 554 : Weight = 3.3296
Node 426 -- Node 487 : Weight = 3.8238
Node 427 -- Node 465 : Weight = 4.4168
Node 428 -- Node 550 : Weight = 2.0364
Node 428 -- Node 536 : Weight = 2.5059
Node 428 -- Node 557 : Weight = 2.5204
Node 428 -- Node 438 : Weight = 2.5205
Node 428 -- Node 511 : Weight = 2.7062
Node 431 -- Node 506 : Weight = 2.1808
Node 431 -- Node 444 : Weight = 2.2488
Node 431 -- Node 458 : Weight = 2.7726
Node 432 -- Node 547 : Weight = 2.6526
Node 433 -- Node 508 : Weight = 1.7494
Node 433 -- Node 483 : Weight = 1.9592
Node 433 -- Node 485 : Weight = 2.3249
Node 434 -- Node 508 : Weight = 2.4012

Node 435 -- Node 490 : Weight = 2.7399
Node 437 -- Node 549 : Weight = 5.1544
Node 438 -- Node 444 : Weight = 1.9066
Node 439 -- Node 469 : Weight = 4.8460
Node 440 -- Node 536 : Weight = 2.4872
Node 441 -- Node 466 : Weight = 3.2898
Node 442 -- Node 547 : Weight = 2.3471
Node 442 -- Node 561 : Weight = 2.5537
Node 442 -- Node 529 : Weight = 2.7840
Node 442 -- Node 503 : Weight = 3.0673
Node 444 -- Node 465 : Weight = 2.0991
Node 445 -- Node 503 : Weight = 3.2140
Node 446 -- Node 556 : Weight = 1.8492
Node 448 -- Node 501 : Weight = 2.1982
Node 448 -- Node 456 : Weight = 2.4154
Node 448 -- Node 510 : Weight = 2.4698
Node 449 -- Node 513 : Weight = 3.7041
Node 450 -- Node 529 : Weight = 3.4025
Node 450 -- Node 568 : Weight = 3.5000
Node 453 -- Node 539 : Weight = 2.0441
Node 454 -- Node 515 : Weight = 7.0626
Node 456 -- Node 546 : Weight = 2.7464
Node 457 -- Node 517 : Weight = 2.9626
Node 459 -- Node 512 : Weight = 2.8588
Node 460 -- Node 473 : Weight = 2.0349
Node 461 -- Node 505 : Weight = 1.8687
Node 462 -- Node 564 : Weight = 3.8011
Node 463 -- Node 552 : Weight = 3.8707
Node 466 -- Node 560 : Weight = 3.0517
Node 466 -- Node 519 : Weight = 3.1100
Node 467 -- Node 544 : Weight = 8.2685
Node 469 -- Node 522 : Weight = 4.0466
Node 472 -- Node 476 : Weight = 3.1000
Node 472 -- Node 495 : Weight = 4.3338
Node 474 -- Node 555 : Weight = 2.3988
Node 476 -- Node 490 : Weight = 2.6276
Node 477 -- Node 513 : Weight = 2.7701
Node 477 -- Node 559 : Weight = 2.8675
Node 478 -- Node 518 : Weight = 2.1767
Node 479 -- Node 540 : Weight = 3.4316
Node 480 -- Node 536 : Weight = 2.2211
Node 480 -- Node 542 : Weight = 2.9954
Node 481 -- Node 561 : Weight = 3.4809
Node 481 -- Node 564 : Weight = 3.4835
Node 483 -- Node 505 : Weight = 1.4194
Node 484 -- Node 500 : Weight = 2.0238
Node 484 -- Node 524 : Weight = 2.1829
Node 488 -- Node 504 : Weight = 5.2469
Node 490 -- Node 540 : Weight = 2.6407
Node 491 -- Node 497 : Weight = 1.6077
Node 492 -- Node 549 : Weight = 2.8296
Node 496 -- Node 521 : Weight = 3.0462
Node 496 -- Node 503 : Weight = 3.1119
Node 496 -- Node 553 : Weight = 3.6417
Node 499 -- Node 551 : Weight = 2.8378

```
Node 500 -- Node 547 : Weight = 2.1513
Node 503 -- Node 533 : Weight = 3.5790
Node 504 -- Node 564 : Weight = 3.2162
Node 505 -- Node 539 : Weight = 1.6334
Node 505 -- Node 557 : Weight = 1.7933
Node 507 -- Node 529 : Weight = 3.8115
Node 511 -- Node 566 : Weight = 3.3898
Node 512 -- Node 536 : Weight = 2.6663
Node 513 -- Node 565 : Weight = 3.9400
Node 516 -- Node 559 : Weight = 3.6886
Node 517 -- Node 543 : Weight = 3.1795
Node 518 -- Node 524 : Weight = 2.1639
Node 518 -- Node 541 : Weight = 2.5113
Node 520 -- Node 553 : Weight = 3.9000
Node 521 -- Node 559 : Weight = 2.4507
Node 523 -- Node 558 : Weight = 2.8544
Node 527 -- Node 536 : Weight = 3.9510
Node 531 -- Node 557 : Weight = 1.8009
Node 535 -- Node 547 : Weight = 2.8183
Node 538 -- Node 567 : Weight = 2.5599
Node 544 -- Node 553 : Weight = 12.2999
Node 547 -- Node 551 : Weight = 2.0340
Node 557 -- Node 558 : Weight = 1.9487

Total MST Weight: 1393.8521
```

Task 6: Implement Graph-Based Clustering ($K=2$)

- Sort the edges of the MST in descending order by edge weight.
- Remove the largest $K - 1$ edge (1 largest edge for 2 clusters) to split the MST into 2 connected components.
- Assign final cluster labels based on the resulting component indices.

In [9]:
```python
# Task 6: Implement Graph-Based Clustering (K = 2)

# Set number of clusters
K = 2

# Create a copy of the MST
cluster_graph = MST.copy()

# Sort MST edges by weight in descending order
sorted_edges = sorted(
    cluster_graph.edges(data=True),
    key=lambda x: x[2]['weight'],
    reverse=True
)

# Remove the largest K-1 edges
```

```python
edges_to_remove = sorted_edges[:K-1]

for u, v, data in edges_to_remove:
    cluster_graph.remove_edge(u, v)

# Find connected components
components = list(nx.connected_components(cluster_graph))

# Assign cluster labels
cluster_labels = np.zeros(len(X_scaled), dtype=int)

for cluster_id, component in enumerate(components):
    for node in component:
        cluster_labels[node] = cluster_id

# Display results
print("Number of Clusters:", len(components))

print("\nCluster Sizes:")
for cluster_id in range(len(components)):
    print(
        f"Cluster {cluster_id}: "
        f"{np.sum(cluster_labels == cluster_id)} samples"
    )

# Display the removed edge
print("\nRemoved Edge:")
for u, v, data in edges_to_remove:
    print(
        f"Node {u} -- Node {v} : "
        f"Weight = {data['weight']:.4f}"
    )

# Display first 20 cluster labels
print("\nFirst 20 Cluster Labels:")
print(cluster_labels[:20])
```

```
Number of Clusters: 2

Cluster Sizes:
Cluster 0: 567 samples
Cluster 1: 2 samples

Removed Edge:
Node 544 -- Node 553 : Weight = 12.2999

First 20 Cluster Labels:
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
```

Task 7: valuate MST Clustering Performance

Evaluate cluster assignment against ground truth labels using:

- Silhouette Score
- Adjusted Rand Index (ARI)
- Normalized Mutual Information (NMI)

In [10…

```python
# Task 7: Evaluate MST Clustering Performance

from sklearn.metrics import (
    silhouette_score,
    adjusted_rand_score,
    normalized_mutual_info_score
)

# Convert diagnosis labels into numerical labels
# B = 0, M = 1
true_labels = df['y'].map({'B': 0, 'M': 1}).values

# Check diagnosis label conversion
if pd.isna(true_labels).any():
    raise ValueError(
        "Diagnosis labels must be B or M. "
        "Check the unique values in df['y']."
    )

# 1. Silhouette Score
silhouette = silhouette_score(X_scaled, cluster_labels)

# 2. Adjusted Rand Index (ARI)
ari = adjusted_rand_score(true_labels, cluster_labels)

# 3. Normalized Mutual Information (NMI)
nmi = normalized_mutual_info_score(
    true_labels, cluster_labels
)

# Display results
print("MST Clustering Performance")
print("--------------------------")
print("Silhouette Score:", round(silhouette, 4))
print("Adjusted Rand Index (ARI):", round(ari, 4))
print("Normalized Mutual Information (NMI):", round(nmi, 4))
```

```
MST Clustering Performance
--------------------------
Silhouette Score: 0.6607
Adjusted Rand Index (ARI): 0.0048
Normalized Mutual Information (NMI): 0.0102
```

Task 8: Visualize MST and Graph-Based Clusters

- Select two representative features (e.g., mean radius vs. mean texture) for 2D plotting.
  - Plot 1: Unclustered standardized data points.
- Plot 2: Minimum Spanning Tree (MST) edge connections overlaid on the data.
- Plot 3: Final clusters formed after cutting the $K - 1$ largest edge(s).

In [11…

```python
# Task 8: Visualize MST and Graph-Based Clusters

import matplotlib.pyplot as plt

# Select two representative standardized features
feature_x = 'x.radius_mean'
feature_y = 'x.texture_mean'

# Check that the selected features exist
if feature_x not in X_scaled.columns or feature_y not in X_scaled.columns
    raise ValueError(
        "Required columns not found. Available columns are: "
        + str(X_scaled.columns.tolist())
    )

# Get feature values
x = X_scaled[feature_x].values
y_plot = X_scaled[feature_y].values

# Plot 1: Unclustered Standardized Data
plt.figure(figsize=(8, 6))

plt.scatter(x, y_plot, s=30, alpha=0.7)

plt.xlabel('Mean Radius (Standardized)')
plt.ylabel('Mean Texture (Standardized)')
plt.title('Unclustered Standardized Data')
plt.grid(True, alpha=0.3)

plt.show()

# Plot 2: MST Edge Connections
plt.figure(figsize=(10, 7))
```

```python
# Plot data points
plt.scatter(x, y_plot, s=25, alpha=0.7)

# Draw MST edges
for u, v in MST.edges():
    plt.plot(
        [x[u], x[v]],
        [y_plot[u], y_plot[v]],
        linewidth=0.7,
        alpha=0.5
    )

plt.xlabel('Mean Radius (Standardized)')
plt.ylabel('Mean Texture (Standardized)')
plt.title('Minimum Spanning Tree (MST)')
plt.grid(True, alpha=0.3)

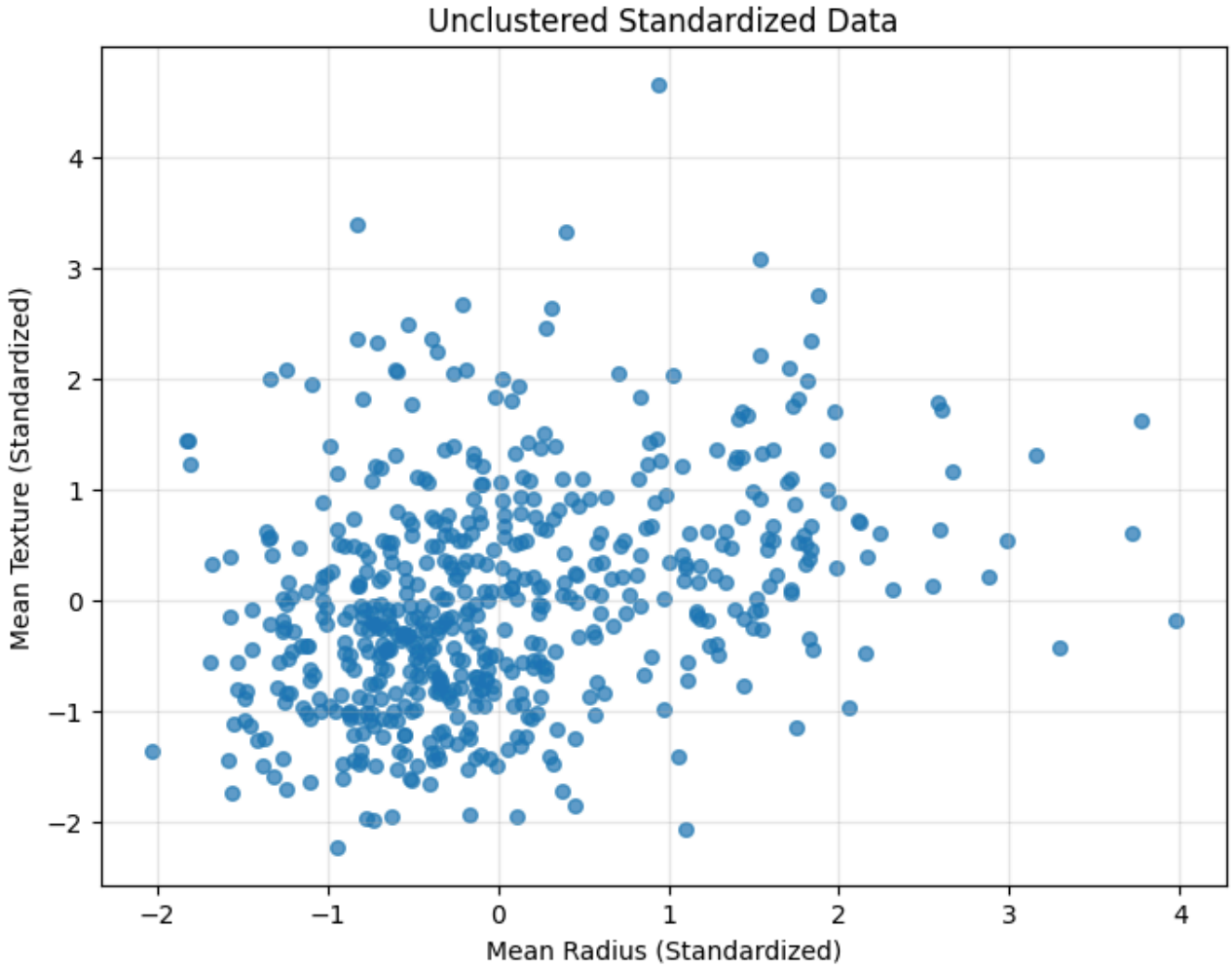
plt.show()

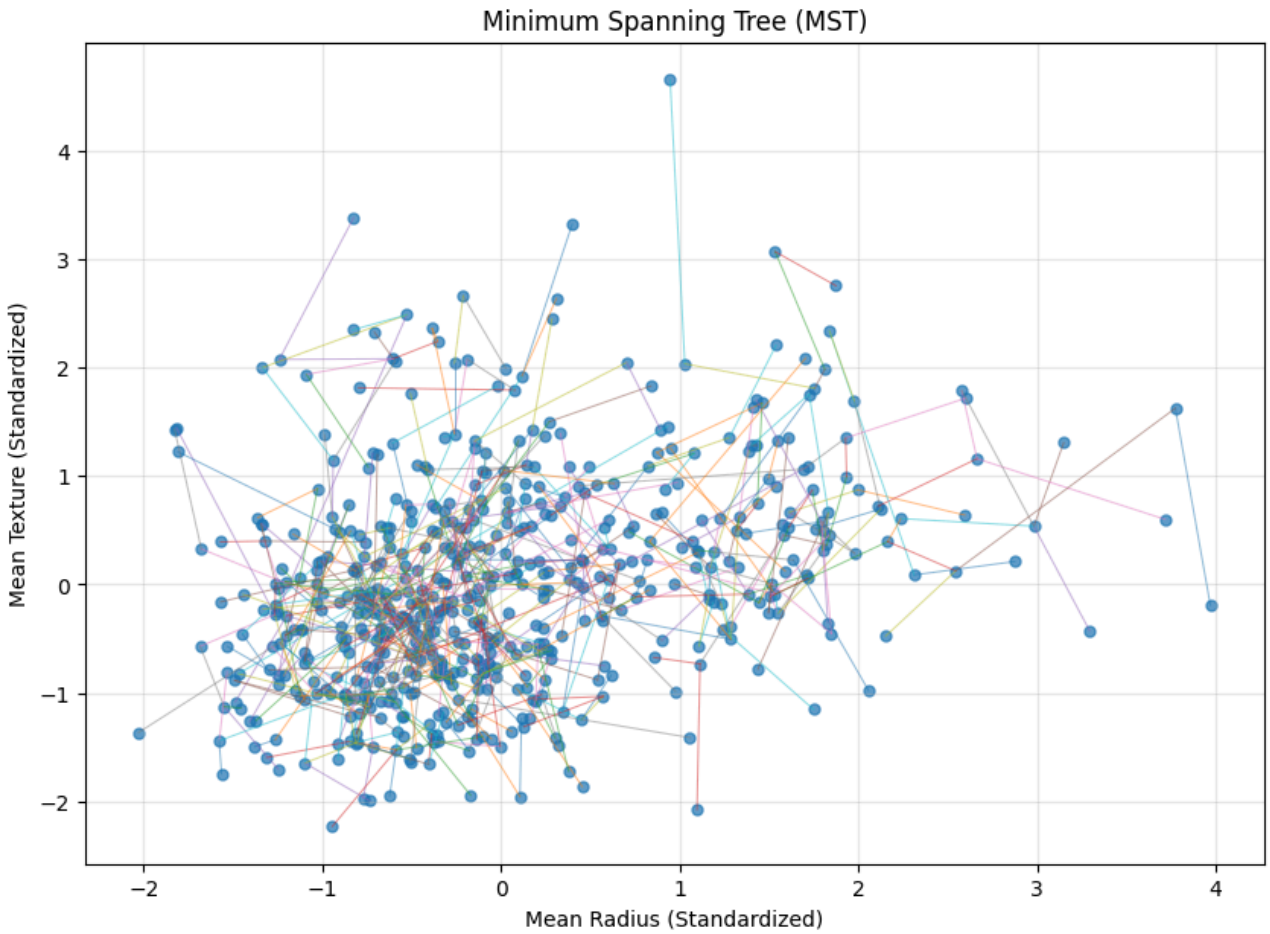
# Plot 3: Final Graph-Based Clusters
plt.figure(figsize=(8, 6))

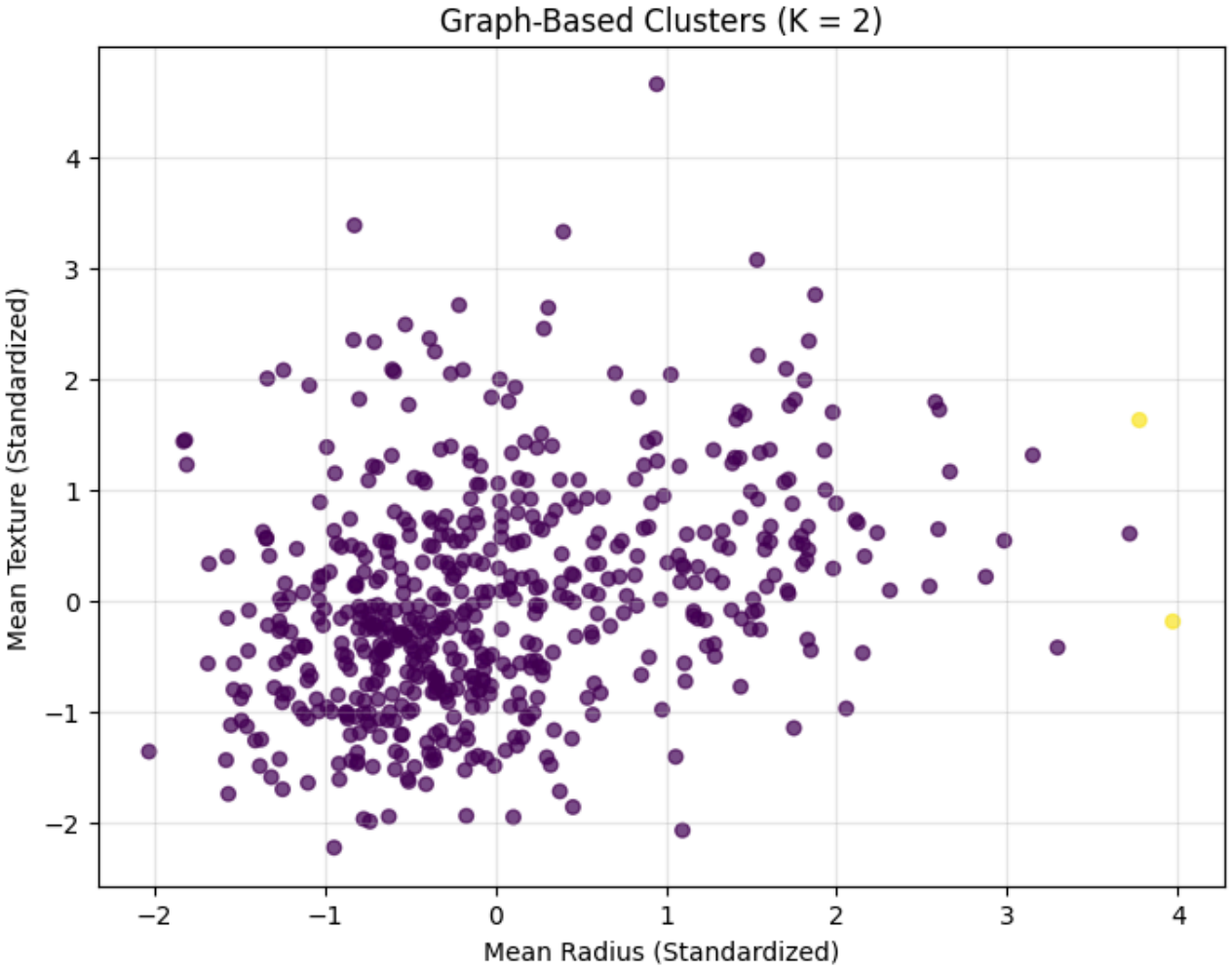
plt.scatter(
    x,
    y_plot,
    c=cluster_labels,
    s=30,
    alpha=0.7
)

plt.xlabel('Mean Radius (Standardized)')
plt.ylabel('Mean Texture (Standardized)')
plt.title('Graph-Based Clusters (K = 2)')
plt.grid(True, alpha=0.3)

plt.show()
```


Unclustered Standardized Data
Mean Texture (Standardized)
Mean Radius (Standardized)
4
3
2
1
0
−1
−2
−2
−1
0
1
2
3
4


Minimum Spanning Tree (MST)
Mean Texture (Standardized)
Mean Radius (Standardized)
4
3
2
1
0
−1
−2
−2
−1
0
1
2
3
4



Conclusion

The experiment successfully demonstrated the implementation of Graph-Based Clustering using a Minimum Spanning Tree (MST) on the Breast Cancer Wisconsin (Diagnostic) dataset. The dataset was preprocessed, cleaned, and standardized before computing pairwise distances. The MST was generated, and by removing the largest edge(s), clusters representing tumor categories were obtained, evaluated using clustering metrics, and visualized.